

LAI - Your Viva Survival Guide

Legal Document Intelligence System · written for **Praveen** · you are **Person 1 - Introduction + Overall Flow** · the easiest, lowest-risk slot. Read top to bottom; the most important things are first.

The one sentence to memorise: "LAI is an AI system that reads a PDF or DOCX contract, uses IBM Docling to parse its structure, Gemini Flash to extract the key legal clauses, a hybrid of rules plus Gemini to score each clause for risk, and then writes a plain-English executive summary - turning an unstructured legal document into structured, interactive legal insights."

Your job in one line: You open the viva. You explain **what the project is** and **how a contract flows through the system end to end**, then hand off to Person 2. You do **not** have to defend deep technical choices - those belong to Persons 2 to 5. There are **no listed counter-questions** for your section. Nail Sections 0, 3, and 4 below and you are done.

CONTENTS

0. The 60-second opener (say this first)
1. Remember only 5 things (the pipeline)
2. The whole system, plainly
3. YOUR script, word for word
4. How a contract flows end to end
5. What your teammates explain
6. The 5 questions everyone must know
7. Tricky questions + safe answers
8. Plain-English glossary
9. Live demo + if you get stuck

0. The 60-second opener (what to say first)

If the examiner says "tell me about your project," say this:

"Our project is **LAI - a Legal Document Intelligence System**. The goal is to **automate contract review using Generative AI**.

Reviewing contracts by hand is slow and expensive - a legal team has to find the risky clauses, compare them to market standards, write a summary, and answer questions about the document. We built a system that does this end to end.

The user uploads a PDF or DOCX contract. It reaches our **FastAPI backend**, which creates a contract record in **PostgreSQL** and kicks off a background pipeline with five stages: **document parsing, clause extraction, risk scoring, executive summary, and database storage**. Once that finishes,

the frontend lets the user read the summary, inspect each clause's risk, compare contracts side by side, and chat with the contract.

So in short, we turn an unstructured legal document into **structured legal insights** that anyone can understand and interact with."

That paragraph alone covers most of what an examiner wants from the opening. Practice it out loud 3 times. Everything else in this guide is backup.

1. If you remember only 5 things (the pipeline)

Your whole section is the **5-stage pipeline**. Memorise these in order - they are the backbone of every answer you give.

1. Parse. IBM Docling reads the PDF/DOCX and keeps its **structure** - headings, section numbers, tables - not just plain text.

2. Extract. Gemini Flash reads the parsed document and pulls out the named **clauses** (indemnity, liability, termination, etc.) as clean **JSON**.

3. Score. Each clause gets a **risk score** from a **hybrid** of regex rules + Gemini, compared against a market-standard baseline.

4. Summarise. Gemini writes a one-page **plain-English executive summary** of the whole contract.

5. Store. Everything is saved in **PostgreSQL** so the frontend can show it instantly and re-use it later.

If you only practise one thing, practise saying these 5 stages smoothly. "Parse, extract, score, summarise, store." That is the heart of Person 1.

2. The whole system, in plain English

Think of LAI as a request that travels from the browser, through the backend pipeline, into the database, and back to the screen.

User uploads a **PDF or DOCX** contract in the browser (React + Vite frontend)

FastAPI backend saves the file, creates a **Contract** row in PostgreSQL with status **uploaded**

A **background pipeline** starts so the upload returns instantly (no waiting, no timeout)

Stage 1 - Docling parses the file into text + markdown + structured JSON

Stage 2 - Gemini extracts the named clauses as JSON

Stage 3 - Hybrid scorer rates each clause's risk (rules + Gemini)

Stage 4 - Gemini writes the plain-English executive summary

Stage 5 - all results saved to PostgreSQL; status becomes **complete**

Frontend shows the **summary, clause risks, compare view, and contract chat**

YOUR PART You own the **overall flow** - the middle 5 stages above. You do not need to defend *how* each stage works internally; that is Persons 2 to 5. You just narrate the journey. That makes you the person who "sees the whole picture," which is exactly what an opener should do.

3. Your script, word for word

This is the polished version of the official Person 1 script. Say it in your own words, but hit every bold phrase.

3.1 What the project is (the problem)

"Our project is **LAI, a Legal Document Intelligence System**. The objective is to **automate contract review using Generative AI**.

Organisations spend a huge amount of time reviewing contracts manually. Legal teams have to **identify risky clauses, compare them against market standards, prepare executive summaries, and answer questions** about the contract. This is slow, expensive, and needs legal expertise. We

built an AI system that handles this end to end."

3.2 What happens when you upload (the flow)

"The user uploads a **PDF or DOCX** contract through the frontend. The request reaches our **FastAPI backend**, where a contract record is first created in **PostgreSQL** with the status set to `uploaded` . After that, a **background processing pipeline** starts."

- 1 **Document Parsing** - the file is converted into structured text.
- 2 **Clause Extraction** - the important legal clauses are pulled out.
- 3 **Risk Scoring** - each clause is scored for risk.
- 4 **Executive Summary Generation** - a one-page plain-English summary is written.
- 5 **Database Storage** - all results are saved.

3.3 What the user can do next (the features) + handover

"Once analysis is complete, the frontend gives several features - **viewing the executive summary**, **checking individual clause risks**, **comparing contracts**, and **chatting with the contract**."

So overall, our system converts an **unstructured legal document into structured legal insights** that are easy to understand and interact with."

Your handover line: "I'll now hand over to **[Person 2]**, who will explain how we convert the uploaded document into structured legal data - the parsing and clause extraction stage."

4. How a contract flows end to end (your strongest answer)

If the examiner says "walk me through what happens when I upload a contract," narrate this. If you can say it smoothly, you have basically passed.

- 1 I upload `contract.pdf` . The frontend sends it to the backend, which **saves the file** and creates a `Contract` row in PostgreSQL with status `uploaded` .
- 2 The backend starts a **background task** and immediately returns, so the browser never hangs -

even a 100-page contract uploads instantly. The status moves `parsing` → `analyzing` → `complete`, and the frontend polls it.

- 3 **Docling parses** the file into raw text, markdown, and a structured JSON tree - keeping section numbers and headings, which matters because clauses reference each other.
- 4 The markdown goes to **Gemini Flash**, which returns the named clauses as **structured JSON** - indemnity, limitation of liability, confidentiality, termination, IP, payment terms, governing law.
- 5 Each clause is **scored**: regex rules catch obvious red flags, Gemini compares it to a market-standard baseline, and we blend them **40% rules + 60% Gemini**. We also compute one **overall contract risk score**.
- 6 Gemini writes the **executive summary**; everything is **saved to PostgreSQL**; status becomes `complete`; the frontend renders the summary, clause cards, compare view, and chat.

Notice steps 2 and 3 quietly answer two favourite examiner questions: **"how do you handle a huge contract without timing out?"** (background task) and **"why keep document structure?"** (cross-references between clauses). Drop those phrases in and you look prepared.

5. What your teammates explain (so you're never blank)

You won't be drilled on these, but if a question drifts to a teammate's area, give the one-paragraph version and pass it over.

Person 2 Document Parsing + Clause Extraction

Docling parses the file while preserving structure (headings, tables, section numbers), then **Gemini Flash with structured JSON output** extracts the named clauses. We use an LLM instead of regex because clauses can be worded in countless ways, and structured JSON because it stores and renders predictably. *If asked*: "Docling keeps the document hierarchy; Gemini understands the legal meaning, not just patterns."

Person 3 Hybrid Risk Scoring + Executive Summary

Two parts: a **deterministic rule engine** (regex for things like "unlimited liability," "sole discretion," "automatic renewal") and **Gemini**, which compares each clause to a market-standard baseline and returns a risk score, explanation, and plain-English summary. Final score = **40% rules + 60%**

Gemini. Then Gemini writes a one-page executive summary. *If asked:* "Rules give deterministic detection; Gemini gives contextual understanding - neither alone is enough."

Person 4 Database + Features

Three PostgreSQL tables: **Contract** (metadata, raw text, overall risk, summary, status), **Clause** (every extracted clause stored separately with its type, risk, deviation, and plain-English summary), and **Baseline** (market-standard templates). Features built on top: executive summary page, compare contracts, and contract chat (which uses **long-context prompting** - full contract + clause metadata + last 5 messages sent to Gemini). *If asked:* "Storing clauses separately avoids re-extracting and enables clause-level comparison."

Person 5 Design Decisions + Tradeoffs + Future

Justifies the choices: Docling for structure, structured JSON for reliability, hybrid scoring for balance, baselines in PostgreSQL, and **long-context prompting instead of RAG** because we analyse a single contract and full context preserves cross-clause relationships. Known limits: no RAG, no vector DB, no persistent chat memory, higher token cost. Future: RAG over a benchmark corpus like **CUAD**, a vector database, cross-contract search, and agentic workflows. *If asked:* "We chose simplicity that fits a single-contract use case, with a clear upgrade path."

6. The 5 questions everyone must know

The team agreed everyone memorises these five, even with the viva split. Learn them cold.

Question	One-line answer
Why Docling?	It preserves headings, tables, lists, and section hierarchy - far better than a plain text parser, and structure matters for legal cross-references.
Why hybrid scoring?	Rules give deterministic detection of obvious risks; Gemini understands legal semantics. Together they are reliable and nuanced.
Why long context instead of RAG?	We analyse a single contract, and the full document fits in Gemini's large context window. Sending everything preserves cross-clause relationships. RAG helps more with large document collections.

Why PostgreSQL?	Contracts, clauses, and baselines have clean relational links, and structured storage makes querying and comparison efficient.
Biggest future improvement?	Replace hand-written baselines with RAG over a benchmark corpus like CUAD for market comparison, then add vector search and agentic workflows.

7. Tricky questions + safe answers

These can come up - including the ones your own teammates raised. Click to expand. Answer calmly in your own words.

- ▶ **Q. What is your project in one line?**
- ▶ **Q. Walk me through what happens when I upload a contract.**
- ▶ **Q. Why didn't you use a graph database for the cross-references?**
- ▶ **Q. Why not use RAG for chat or comparison?**
- ▶ **Q. Which Gemini model, and why Gemini at all?**
- ▶ **Q. Why not a legal-specific model like Legal-BERT or SaulLM?**
- ▶ **Q. How do you handle a 100-page contract without timing out?**
- ▶ **Q. What does "structured output" mean?**
- ▶ **Q. How is the risk score actually calculated?**
- ▶ **Q. Which clause types do you extract?**
- ▶ **Q. Is it deployed? Can you show it live?**
- ▶ **Q. What are the limitations?**

7b. The "what if it were X instead of Y" pushbacks

The tactic: the examiner agrees with your reason, then changes the scenario to see if you understand *when your choice stops being right*. The winning move is to **name the boundary and concede it cleanly** - "yes, in that case the other option wins, and that's exactly why it's on our roadmap." That shows judgement, not stubbornness. Never defend a choice past the point where it's actually correct.

► **Q. You skipped RAG because it's a single contract. What if a user uploads 500 contracts and asks a question across all of them?**

► **Q. What if the contract were so large it didn't fit in the context window?**

► **Q. What if Gemini's context window were small, like 8K tokens - would you still avoid RAG?**

► **Q. You said a graph DB is overkill. What if you had to trace which of 10,000 contracts all reference the same master agreement?**

► **Q. Gemini can read PDFs directly. What if you sent the raw PDF straight to Gemini and skipped Docling?**

► **Q. What if the uploaded PDF is a scanned image with no selectable text?**

► **Q. What if you used only Gemini for scoring - or only the rules? Why blend?**

► **Q. Why 40/60? What if it were 50/50 or 30/70?**

► **Q. What if a contract has one critical clause buried among fifty harmless ones - doesn't the average hide it?**

► **Q. What if you'd used GPT-4o or Claude instead of Gemini?**

► **Q. What if you needed semantic search over clauses later - doesn't that force you off PostgreSQL?**

8. Plain-English glossary

Term	In one line
Clause	A self-contained provision of a contract (e.g. the indemnity clause, the termination clause).
Docling	IBM's open-source parser that turns a PDF/DOCX into structured text while keeping headings, tables, and section numbers.
Gemini Flash	Google's fast, low-cost LLM with a large context window; we use it for extraction, scoring, summary, and chat.
Structured output	Forcing the LLM to answer as JSON that matches a fixed schema, so results are predictable and storable.
Clause extraction	Identifying and pulling out the named legal clauses from the parsed document.
Market-standard baseline	A reference "normal" version of a clause; we compare the contract's clause against it to judge deviation.
Market deviation	How far a clause differs from the baseline: favourable, unfavourable, unusual, or standard.
Hybrid risk scoring	Combining deterministic regex rules with Gemini's judgement (40% / 60%) into one risk score.
Risk level	The score bucketed into low (0-25), medium (26-50), high (51-75), or critical (76-100).
Risk category	The kind of risk: financial, operational, legal, or reputational.
Executive summary	A one-page plain-English overview: what the contract covers, who carries risk, top issues to negotiate.
Long-context prompting	Sending the entire contract in one prompt instead of retrieving snippets - works because the context window is large.
RAG	Retrieval-Augmented Generation: search a big corpus for relevant chunks, then feed them to the LLM. We don't use it (single contract), but it's our future plan.
Vector database	A store for embeddings that enables semantic search; part of the future RAG roadmap.
Background task	Work the backend runs after responding to the request, so the upload doesn't block or time out.
FastAPI	The Python web framework serving our backend API.

PostgreSQL	Our relational database holding the Contract, Clause, and Baseline tables.
CUAD	A public benchmark dataset of annotated contracts - the corpus we'd use for future RAG-based market comparison.

9. Live demo + if you get stuck

If asked to show it working, this runs the full stack locally (from the repo root):

```
docker compose up -d                                # PostgreSQL
cd backend && .venv\Scripts\activate
uvicorn app.main:app --reload --port 8000            # backend  -> localhost:8000/docs
cd frontend && npm run dev                             # frontend -> localhost:5173
```

The three "wow" moments: upload a contract and watch the status move to **complete**; open a contract to show the **risk gauge + clause cards**; and the **compare** view showing the same clause across contracts.

Never bluff a wrong detail. If you don't know an exact number, describe the idea. Examiners reward understanding over memorised specifics.

- **Buy time:** "Let me walk through what happens step by step..." then narrate the flow from Section 4. It touches almost everything.
- **Redirect to your strength:** almost any question can be grounded in the 5-stage pipeline - parse, extract, score, summarise, store - which is your area.
- **Hand off cleanly:** if a question goes deep into a teammate's module, give the one-paragraph overview from Section 5, then "that part was owned by [name], who can go deeper."
- **Own the simplifications:** "We deliberately kept it focused on a single-contract use case - that's why long-context instead of RAG - with a clear path to scale." That's a strength, not a weakness.

LAI · Legal Document Intelligence System · FastAPI + React + PostgreSQL + Docling + Gemini Flash. You are Person 1: own Sections 0, 3, and 4, keep "parse, extract, score, summarise, store" on the tip of your tongue, and hand off with a smile.